Lab 1 Report

Tuesday 2:15 - 5:00

Austin Petersen, Dylan Thornburg,
Madeline Follosco

Introduction

This lab will familiarize us with some basic operations of the Arduino. We will demonstrate the functionality of the digital pins by blinking LEDs and using pull-up resistors and switches to read from them. This lab will also familiarize us with basic circuit construction using breadboards, SIP and DIP components, and passive components.

Demo #1

Assignment 1: Basic Introduction to the Arduino Environment

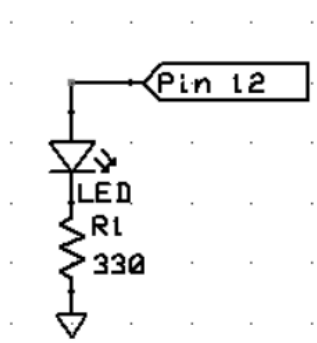
The Arduino IDE should be already installed on your computer

You are using the Arduino Mega 2560. Documentation is here:

Arduino Mega 2560 Home Page and https://docs.arduino.cc/hardware/mega-2560. There is also a datasheet on the Google Drive.

Follow these steps:

1. Run one of the blinking light programs from class to make sure your Arduino is working. Class programs are on the Google Drive in the Class Examples directory.
2. Using the wires and 400-pin protoboard, plug in a single red LED and a resistor as shown in figure below. R1=330 Ohm. Pay attention to direction. Datasheets are available on the Google Drive to document parts.



If you do not know how to read the schematic or identify parts, ask classmates or TA/prof. for help.

Modify one of your programs from class so that it makes the new external LED blink instead of the built-in LED. **(This is DEMO #1)**

Run this program and record your observations.

CODE

```
int ledPin=12;


void setup() {
```

```
  Serial.begin(9600);
  Serial.println("Setup complete.");
  pinMode(ledPin, OUTPUT);
}
int count = 0;

void loop() {
  BlinkOn();
}
void BlinkOn() {
  digitalWrite(ledPin, HIGH);
  delay(500);
  digitalWrite(ledPin, LOW);
  delay(500);
}
```

This program runs and blinks the LED on for 500 ms and off for 500 ms.

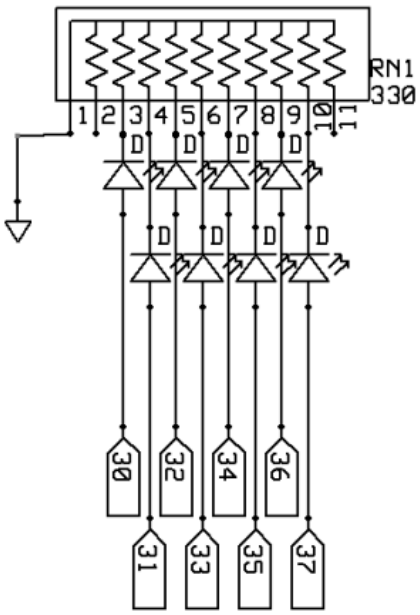
Observations:
A new external LED blinks instead of the built in LED.

Demo #2

Assignment 2

- Connect the middle 8 LEDs on the 10-wide red LED pack to the pins 30-37 of the Arduino with the 11-pin SIP resistor pack. See schematic below.
- Pay attention to which pin is which on the LED pack and resistor pack. If you do not know – consult documentation or design an experiment to measure.
- The sketch you ran before can turn a single LED on or off. Write a program to make the LEDs blink in the following sequence:
  a) Turn one LED on at a time for 0.25 second and then turn it off. Repeat this for the other LEDs so that only one LED is lighted at a time. Light the LEDs from right to left. Then turn all LEDs on together for 0.8 seconds and then turn them off. Repeat indefinitely.

Design two new and different blink sequences for the LEDs.
(This is DEMO #2 – demo all 3 sequences)



CODE

```
int ledPin[8]={2,3,4,5,6,7,8,9};

void setup() {
```

```
  Serial.begin(9600);
  Serial.println("Setup complete.");
  int i;
  for(i=0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
}
int count = 0;

void loop() {
  /* Seq 1*/
  int i;
  delay(250);
  turnOffAll();
  for(i=0; i < 8; i++) BlinkOn(i);
  turnOnAll();
  delay(800);
  turnOffAll();
  delay(2000);

  /* Seq 2*/
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
```

```
  turnOnAll();
  delay(100);
  turnOffAll();
  delay(100);
  delay(2000);

  /* Seq 3*/
  for(i=0; i < 8; i++) BlinkOnRapid(i);
  for(i=8; i > 0; i--) BlinkOnRapid(i-1);
}
void BlinkOn(int num) {
  digitalWrite(ledPin[num], HIGH);
  delay(250);
  digitalWrite(ledPin[num], LOW);
  delay(250);
}
void turnOnAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], HIGH);
}
void turnOffAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], LOW);
}

void BlinkOnRapid(int num) {
  digitalWrite(ledPin[num], HIGH);
  delay(100);
  digitalWrite(ledPin[num], LOW);
  delay(10);
}
```

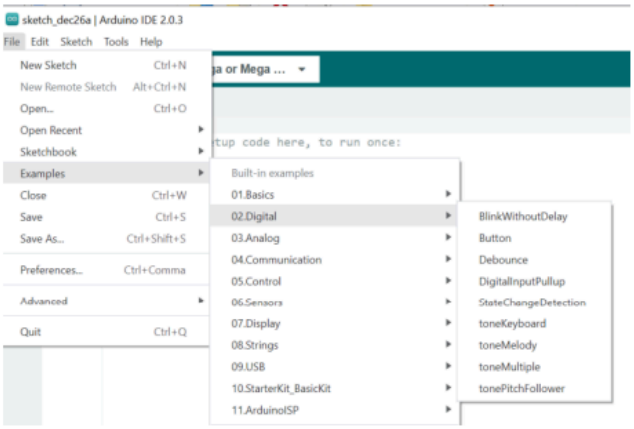
Observations:
All 3 sequences worked as expected. For the 2 new ones created, we had one sequence blink all lights on and off with a delay in between, and another where each light turned on one at a time with delay from one end to the other.

Demo #3

Part 2: Making timing intervals in the Arduino Environment

The `delay()` function was used in Blink to define a time interval. An alternative way to define time is to use the `millis()` function. `millis()` reads a hardware timer that counts in miliseconds.

Load the program called BlinkWithoutDelay as shown below




```
/*
  Blink without Delay

  Turns on and off a light emitting diode (LED) connected to a digital pin,
  without using the delay() function. This means that other code can run at the
  same time without being interrupted by the LED code.
```

…

The program above will do the same job as a simple blink program.

Run this program, review the code, and record your observations – put them in your report.

Assignment 3:

- In lab – rewrite the first program from assignment 2 (above) using the `millis()` function instead of `delay()`. **(This is DEMO #3)**
- In your lab report explain: What is the advantage of the using the `millis()` function instead of the `delay()` function?

CODE

```
/* OLD CODE | BAD */
```

```
int ledPin[8]={2,3,4,5,6,7,8,9};
unsigned long timeCheck = 0;
void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  int i;
  for(i=0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
}
int count = 0;

void loop() {
  int i;
  delay(250);
  turnOffAll();
  for(i=0; i < 8; i++) BlinkOn(i);
  turnOnAll();
  while (!(millis()-timeCheck >= 800));
  turnOffAll();
}
void BlinkOn(int num) {
  digitalWrite(ledPin[num], HIGH);
  while (!(millis()-timeCheck >= 250));
  digitalWrite(ledPin[num], LOW);
  while (!(millis()-timeCheck >= 500));
  timeCheck=millis();
}
void turnOnAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], HIGH);
  timeCheck=millis();
}
void turnOffAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], LOW);
  timeCheck=millis();
}
```

```
/* NEW CODE */
int ledPin[8]={2,3,4,5,6,7,8,9};
unsigned long timeCheck = 0;
void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  int i;
  for(i=0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
  turnOffAll();
  timeCheck = millis();
}
int count = 0;
int i=0;
int flag = 0;
void loop() {
  if((millis()-timeCheck) >= 250 && i < 8 && !flag){
    if ((millis()-timeCheck) >= 500) {
      BlinkOff (i++);
      timeCheck = millis();
    }
    else {
      BlinkOn (i);
    }
  }
  if(i >= 8 && !flag) {
    turnOnAll();
    timeCheck = millis();
    flag=1;
  }
  if(flag && (millis() - timeCheck) >=800) {
    turnOffAll();
    i=0;
    flag=0;
    timeCheck=millis();
  }
```

```
 // for(i=0; i < 8; i++) BlinkOn(i);
 // if (double((millis()-timeCheck) % 4000) == 0) turnOnAll();
 // if (double((millis()-timeCheck) % 4800) == 0) turnOffAll();
}

void BlinkOn(int num) {
  digitalWrite(ledPin[num], HIGH);
}
void BlinkOff(int num) {
  digitalWrite(ledPin[num], LOW);
}
void turnOnAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], HIGH);
}
void turnOffAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], LOW);
}
```

Observations:

For the blink without delay example, it functions identically to the blink example but using the millis() function instead of delay, freeing up the board to run in between time checks.

LED's light up one at a time like they were before (lighting up for 0.25 seconds turning off 0.25 seconds one at a time until all have lit up once, then make all of them light up for 0.8 seconds then turn them all off) using milis() in if statements to ensure a non-blocking structure.

Questions: What is the advantage of the using the millis() function instead of the delay() function?

Using delay will "stall" the chip until the value is reached, whereas using millies will allow for other code to run during our waiting period.

milis() allows for time based triggered events (acts as program timer) without delay in your program

We had two version of the millis() implementation. The old code was essentially using the millis() function as a delay and blocking since no code could run outside the while loop. Now we've changed logic using flags and if statements instead to allow for non-blocking structure.

Demo #4

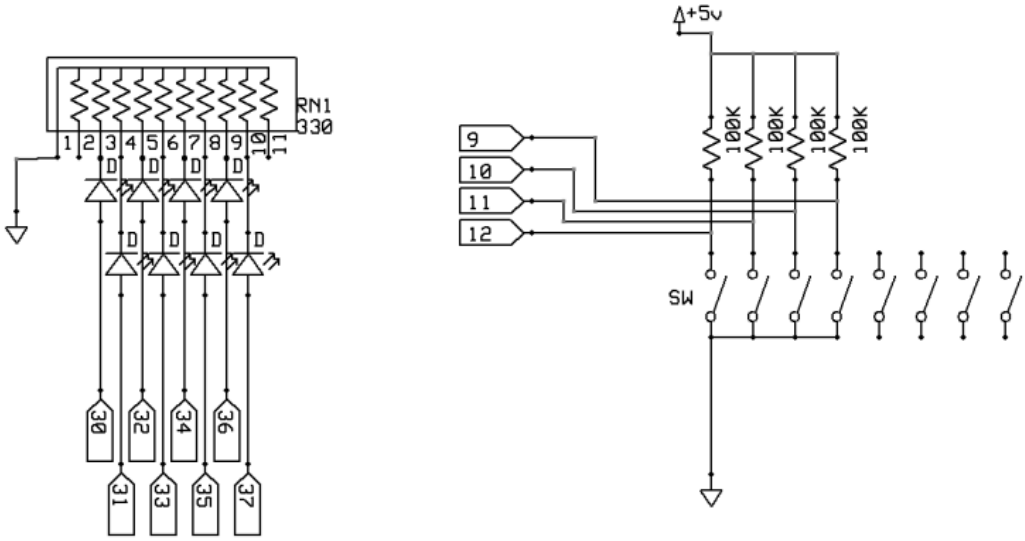
Part 3: Working with Digital I/O

This part of the lab shows how to:

i. address digital ports
ii. output digital signals
iii. read digital signals

Assignment 4:

- Using the 330 Ohm resistors (11-pin SIP), the 10-wide LED, Four 100KΩ resistors and an 8-wide DIP switch create the circuit below. (The LEDs should be the same as before). Be careful to wire the switches exactly as shown.



- Use the function digitalRead() to read the data from the dip-switch. An example showing how to use digitalRead is shown here:

  http://arduino.cc/en/Reference/digitalRead

- Write a program so that based on the dip-switch settings, the first 4 LEDs will show the settings of the 4 DIP switches and the last 4 LEDs will flash the settings of the 4 DIP switches on and off at 500 millisecond intervals. Pay attention to the on/off markings on the switch.
- Switch on = light on/blinking; switch off = light off. **(This is DEMO #4)**

CODE

```
int ledPin[8]={2,3,4,5,6,7,8,9};
int switchPin[4] = {26,28,30,32};
int data[4];
unsigned long timeCheck = 0;
void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  int i;
  for(i=0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
  for(i=0; i < 4; i++) pinMode(switchPin[i], INPUT);
  turnOffAll();
  timeCheck=millis();
}
int count = 0;

void loop() {
  int i;
  ReadWriteFourLights();
  if(millis()-timeCheck >= 500) {
    if (millis()-timeCheck >= 1000) {
      FourLightsOff();
      timeCheck = millis();
    }
    else FourLightsOn();
  }
}

void FourLightsOn() {
  int i;
```

```
  for(i=4; i < 8; i++) digitalWrite(ledPin[i], data[i-4]);
}

void FourLightsOff() {
  int i;
  for(i=4; i < 8; i++) digitalWrite(ledPin[i], LOW);
}

void ReadWriteFourLights(){
  int i;
  for(i=0; i < 4; i++) data[i] = !(digitalRead(switchPin[i]));
  for(i=0; i < 4; i++) digitalWrite(ledPin[i], data[i]);
}

void turnOffAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], LOW);
}
```

Observations:
First 4 leds reflect the settings of the first 4 switches while the last 4 blink on and off the settings of the first 4 switches. We noticed that we had to negate the actual data in code as what was read from the voltages was inverted due to the pull-up resistor design.

Demo #5

This sketch prints sequential numbers on the Serial Monitor. You can try it if you want.

```
/*
* SerialOutput sketch
* Print numbers to the serial port
*/
void setup()
{
Serial.begin(9600); // send and receive at 9600 baud
}
int number = 0;
void loop()
{
Serial.print("The number is ");
Serial.println(number); // print the number
delay(500); // delay half second between numbers
number++; // to the next number
}
```

Click the Serial Monitor icon in the IDE and you should see the output displayed as follows:

The number is 0
The number is 1
The number is 2

- Write a program to read the dip-switch settings and display them on the Serial Monitor. On each line display 4 digits representing the 4 switches. A 0 for each switch that is off and a 1 for each switch is on. Think hard about what voltage gets measured at each Arduino pin when the switch is off and what voltage gets measured when the switch is on.

(This is DEMO #5)

CODE

```
int ledPin[8]={2,3,4,5,6,7,8,9};
int switchPin[4] = {26,28,30,32};
int data[4];
unsigned long timeCheck = 0;
void setup() {
  Serial.begin(9600);
  Serial.println("Setup complete.");
  int i;
  for(i=0; i < 8; i++) pinMode(ledPin[i], OUTPUT);
  for(i=0; i < 4; i++) pinMode(switchPin[i], INPUT);
  turnOffAll();
  timeCheck=millis();
```

```
}
int count = 0;

void loop() {
  int i;
  ReadWriteFourLights();
  if(millis()-timeCheck >= 500) {
    if (millis()-timeCheck >= 1000) {
      FourLightsOff();
      timeCheck = millis();
    }
    else FourLightsOn();
  }
}

void FourLightsOn() {
  int i;
  for(i=4; i < 8; i++) digitalWrite(ledPin[i], data[i-4]);
}

void FourLightsOff() {
  int i;
  for(i=4; i < 8; i++) digitalWrite(ledPin[i], LOW);
}

int temp [4] = {5,5,5,5};
void ReadWriteFourLights(){
  int i;
  for(i=0; i < 4; i++) data[i] = !(digitalRead(switchPin[i]));
  for(i=0; i < 4; i++) {
    if(temp[i] != data[i]){
      temp[i] = data[i];
      if(temp[3] !=5) {
        Serial.print(String(temp[0]) + ", ");
        Serial.print(String(temp[1]) + ", ");
        Serial.print(String(temp[2]) + ", ");
        Serial.println(String(temp[3]));
      }
    }
```

```
  }
  for(i=0; i < 4; i++) digitalWrite(ledPin[i], data[i]);
}

void turnOffAll(){
  int i;
  for(i=0; i < 8; i++) digitalWrite(ledPin[i], LOW);
}
```

Observations:
Everything works as expected. We print the default switch states, and on any state change (when any switch is flipped) we print the current state to the serial monitor.

Conclusion

In this lab we worked with the Arduino, resistors (330ohm and 100KOhm), 11-pin SIP resistor pack, 10-wide red LED pack, and the dip switch. We learned the difference between delay() and millis(). We also read data from the dip-switch using digitalRead() and wrote data to the serial monitor with digitalWrite().